

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 6

Виконав:

студент групи кн-25-1

Рафієв Б. С.

Перевірив:

доцент кафедри КІЕ

Сидоренко В.М.

Кременчук 2026

Тема. Графи. Найкоротші шляхи

Мета: набути практичних навичок розв'язання задач пошуку найкоротших шляхів у графі та оцінювання їх асимптотичної складності.

ХІД РОБОТИ

Завдання для самостійного розв'язання: Пошук найкоротших шляхів у зваженому графі

1. Умова задачі: Задано зважений орієнтований граф, який містить наступні вершини та ребра з відповідними вагами (довжинами):

- Множина вершин: $V = \{1, 2, 3, 4, 5, 6\}$
- Множина орієнтованих ребер: $E = \{(1,2)=7, (1,3)=3, (1,4)=1, (3,4)=7, (3,5)=11, (4,5)=5, (4,6)=4, (4,2)=2, (5,6)=6, (6,2)=8\}$

Необхідно знайти найкоротші відстані від початкової вершини 1 до всіх інших вершин графа, використовуючи покроковий аналіз матриці відстаней за алгоритмом Флойда-Воршалла.

Алгоритм Флойда-Воршалла будує послідовність матриць відстаней $D^{(0)}$, $D^{(1)}$, ..., $D^{(6)}$ розміром 6×6 , де елемент d_{ij} позначає найкоротший шлях від вершини i до вершини j . На кожному кроці k ми перевіряємо, чи можна покращити шлях між будь-якими вершинами i та j , пройшовши через проміжну вершину k за формулою: $d_{ij} = \min(d_{ij}, d_{ik} + d_{kj})$.

Крок 0. Початкова матриця $D^{(0)}$. Заповнюється безпосередніми вагами ребер. По діагоналі стоять 0 (відстань від вершини до самої себе). Якщо прямого ребра немає, фіксується нескінченність (∞).

0	7	3	1	∞	∞
∞	0	∞	∞	∞	∞
∞	∞	0	7	11	∞
∞	2	∞	0	5	4

∞	∞	∞	∞	0	6
∞	8	∞	∞	∞	0

Крок 1. Можливе покращення шляхів через проміжну вершину 1 ($k=1$). Оскільки до вершини 1 не входить жодне ребро з інших вершин, жоден існуючий шлях через неї покращити неможливо. Матриця $D^{(1)}$ залишається ідентичною до $D^{(0)}$.

Крок 2. Можливе покращення шляхів через проміжну вершину 2 ($k=2$). З вершини 2 немає вихідних ребер до інших вузлів (окрім нуля до самої себе). Оновлень у матриці $D^{(2)}$ немає.

Крок 3. Можливе покращення шляхів через проміжну вершину 3 ($k=3$). Перевіряємо транзитні шляхи через вузол 3.

- Знайдено шлях від вершини 1 до вершини 4: $d_{14} = \min(d_{14}, d_{13} + d_{34}) = \min(1, 3 + 7) = 1$ (старий шлях кращий).
- Знайдено шлях від вершини 1 до вершини 5: $d_{15} = \min(\infty, d_{13} + d_{35}) = \min(\infty, 3 + 11) = 14$. Відстань до вершини 5 оновлюється з ∞ на 14.

Крок 4. Можливе покращення шляхів через проміжну вершину 4 ($k=4$). Вершина 4 має багато зв'язків, тому відбувається суттєва оптимізація матриці $D^{(4)}$:

- Шлях $1 \rightarrow 4 \rightarrow 2$: $d_{12} = \min(7, d_{14} + d_{42}) = \min(7, 1 + 2) = 3$. Відстань від 1 до 2 зменшується з 7 до 3.
- Шлях $1 \rightarrow 4 \rightarrow 5$: $d_{15} = \min(14, d_{14} + d_{45}) = \min(14, 1 + 5) = 6$. Відстань від 1 до 5 покращується з 14 до 6.
- Шлях $1 \rightarrow 4 \rightarrow 6$: $d_{16} = \min(\infty, d_{14} + d_{46}) = \min(\infty, 1 + 4) = 5$.
- Шлях $3 \rightarrow 4 \rightarrow 2$: $d_{32} = \min(\infty, d_{34} + d_{42}) = \min(\infty, 7 + 2) = 9$.
- Шлях $3 \rightarrow 4 \rightarrow 5$: $d_{35} = \min(11, d_{34} + d_{45}) = \min(11, 7 + 5) = 11$ (без змін).
- Шлях $3 \rightarrow 4 \rightarrow 6$: $d_{36} = \min(\infty, d_{34} + d_{46}) = \min(\infty, 7 + 4) = 11$.

Крок 5. Можливе покращення шляхів через проміжну вершину 5 ($k=5$).

Через вершину 5 перевіряємо виходи на вершину 6:

- Шлях $1 \rightarrow 5 \rightarrow 6$: $d_{16} = \min(5, d_{15} + d_{56}) = \min(5, 6 + 6) = 5$ (без змін).
- Шлях $3 \rightarrow 5 \rightarrow 6$: $d_{36} = \min(11, d_{35} + d_{56}) = \min(11, 11 + 6) = 11$ (без змін).

Крок 6. Можливе покращення шляхів через проміжну вершину 6 ($k=6$).

Перевіряємо транзит через вершину 6, яка веде до вершини 2 (вага 8):

- Шлях $4 \rightarrow 6 \rightarrow 2$: $d_{42} = \min(2, d_{46} + d_{62}) = \min(2, 4 + 8) = 2$ (без змін).
- Шлях $1 \rightarrow 6 \rightarrow 2$: $d_{12} = \min(3, d_{16} + d_{62}) = \min(3, 5 + 8) = 3$ (без змін).

Розрахунки завершено, матриця стабілізувалася.

Остаточна матриця найкоротших відстаней

Нижче наведено фінальні значення найкоротших відстаней між усіма парами вершин (елементи з нескінченністю ∞ означають повну відсутність орієнтованого маршруту між відповідними вузлами):

0	3	3	1	6	5
∞	0	∞	∞	∞	∞
∞	9	0	7	11	11
∞	2	∞	0	5	4
∞	14	∞	∞	0	6
∞	8	∞	∞	∞	0

2. Оцінка асимптотичної складності алгоритму:

- **Часова складність:** Становить $O(|V|^3)$, де $|V| = 6$ – кількість вершин у графі. Ця кубічна складність зумовлена наявністю трьох вкладених циклів (перебір проміжних вершин k , початкових точок i та кінцевих точок j), які виконуються фіксовану кількість разів незалежно від щільності або структури ребер графа.

- **Просторова складність:** Становить $O(|V|^2)$, оскільки для роботи алгоритму та збереження проміжних і фінальних результатів вимагається виділення пам'яті під двовимірну матрицю відстаней розміром 6×6 .

Висновки: Під час виконання цієї практичної роботи я закріпив практичні навички роботи зі зваженими орієнтованими графами та опанував метод динамічного програмування на прикладі алгоритму Флойда-Воршалла. Було детально вивчено процес розрахунку та оптимізації матриці найкоротших відстаней між усіма парами вершин графа. Теоретичний аналіз та порівняння часових характеристик алгоритмів Дейкстри, Белмана-Форда та Флойда-Воршалла дозволили дійти висновку, що для задач пошуку «всі до всіх» на невеликих графах кубічна складність алгоритму Флойда-Воршалла є цілком виправданою через простоту та комплексність отриманого результату, тоді як для великих систем маршрутизації раціональніше використовувати багаторазовий запуск алгоритму Дейкстри.

Відповіді на контрольні запитання

1. Граф – це абстрактна математична структура та модель, яка використовується для відображення зв'язків між об'єктами, де головними складовими елементами структури є множина вершин (або вузлів) V , які уособлюють самі об'єкти, та множина ребер (або дуг) E , що з'єднують ці вершини між собою. Залежно від наявності визначеного напрямку руху ребра можуть бути орієнтованими чи неорієнтованими, а для відображення вартості чи довжини переходу їм часто присвоюють третій компонент – числовий показник ваги.

2. Для розв'язання цієї задачі, залежно від її специфіки (пошук від одного джерела чи між усіма парами вершин) та властивостей графа, використовують такі класичні алгоритми, як алгоритм Дейкстри для графів без ребер з від'ємною вагою, алгоритм Белмана-Форда для універсального пошуку в умовах наявності від'ємних ваг, алгоритм Флойда-Воршалла для комплексного знаходження матриці відстаней між усіма парами вузлів одночасно, а також алгоритм A^* (A-star) для евристичного прискорення пошуку в просторових та навігаційних задачах.

3. Алгоритм Дейкстри працює за жадібним принципом: він покроково вилучає з черги з пріоритетами вершину з найменшою поточною відстанню від джерела та релаксує всі вихідні ребра, оновлюючи через них найкоротші шляхи до сусідніх вузлів. Головною його особливістю та обмеженням є висока швидкість роботи $O((|V| + |E|) \log V)$ на розріджених графах за умови абсолютної відсутності ребер з від'ємною вагою, оскільки поява від'ємної вартості ламає жадібну логіку та призводить до некоректних результатів.

4. Алгоритм Белмана-Форда – це універсальний алгоритм пошуку найкоротших шляхів від однієї вершини до всіх інших, який ітеративно (рівно $|V| - 1$ разів) перебирає абсолютно всі ребра графа і релаксує відстані до вершин. Його варто застосовувати тоді, коли граф містить ребра з від'ємними вагами, або коли необхідно гарантовано виявити наявність так званого «від'ємного циклу» (циклу, обхід якого нескінченно зменшує довжину шляху), що робить його незамінним у фінансовому моделюванні чи складній мережевій маршрутизації.

5. Алгоритм Флойда–Воршалла базується на динамічному програмуванні та працює шляхом послідовного потрійного циклу, який оцінює можливість покращення (скорочення) шляху між будь-якою парою вершин і та j через кожную проміжну вершину k графа. Головною перевагою алгоритму є знаходження найкоротших відстаней відразу між усіма парами вершин за допомогою дуже простого у реалізації коду, що допускає від'ємні ваги, а ключовими недоліками є висока кубічна часова складність $O(|V|^3)$ та значні витрати пам'яті $O(|V|^2)$, що обмежує його використання виключно невеликими за розміром графами. Шляхи оптимізації алгоритмів: Для оптимізації алгоритму Кана слід використовувати низькорівневі масиви замість важких об'єктів на кшталт `set`, замінити чергу на швидкий двосторонній список `deque` та зберігати граф у вигляді компактного списку суміжності для швидкого доступу до пам'яті. В той час як для оптимізації DFS найефективнішим є повна відмова від рекурсії на користь ітеративного підходу з використанням власного явного стека у пам'яті, а також заміна динамічних структур відстеження відвіданих вершин на прості масиви прапорців фіксованого розміру.